Name: D. Bala Charan ID number: 2520090087

Week 2

SESSION-3

Task 1: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality

```
root@DUDDUKURIs: ~/skill/w
root@DUDDUKURIs:~# cd skill/week-3
root@DUDDUKURIs:~/skill/week-3# nano main.c
root@DUDDUKURIs:~/skill/week-3# gcc main.c -o main
root@DUDDUKURIs:~/skill/week-3#
```

CODE:

```
root@DUDDUKURIs: ~/skill/w
 GNU nano 7.2                                                    main.c *
#include <stdio.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

#define MAX_BUFFER 100
#define MAX_HISTORY 10

char history[MAX_HISTORY][MAX_BUFFER];
int history_count = 0;
int history_index = 0;

void enable_raw_mode(struct termios *old) {
    struct termios raw;

    tcgetattr(STDIN_FILENO, old);
    raw = *old;

    raw.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &raw);
}

void restore_mode(struct termios *old) {
    tcsetattr(STDIN_FILENO, TCSANOW, old);
}

void save_command(char *command) {
    if (strlen(command) == 0)
        return;

    if (history_count < MAX_HISTORY) {
        strcpy(history[history_count], command);
        history_count++;
    } else {
```

```
        for (int i = 1; i < MAX_HISTORY; i++)
            strcpy(history[i - 1], history[i]);

        strcpy(history[MAX_HISTORY - 1], command);
    }

    history_index = history_count;
}

int main() {
    struct termios old;
    char buffer[MAX_BUFFER];
    int index = 0;
    int ch;

    printf("Session 3 - Command History\n");
    printf("Type commands. Use UP/DOWN arrows for history.\n");
    printf("Press Ctrl+C to stop.\n\n");

    enable_raw_mode(&old);

    while (1) {
        index = 0;
        buffer[0] = '\0';

        printf("myshell> ");
        fflush(stdout);

        while (1) {
            ch = getchar();
```

```
            /* Enter key */
            if (ch == '\n' || ch == '\r') {
                buffer[index] = '\0';

                printf("\n");

                if (strcmp(buffer, "exit") == 0) {
                    restore_mode(&old);
                    printf("Exiting...\n");
                    return 0;
                }

                if (strlen(buffer) > 0) {
                    printf("Command: %s\n", buffer);
                    save_command(buffer);
                }

                break;
            }
```

```c
/* Backspace */
if (ch == 127 || ch == 8) {
    if (index > 0) {
        index--;
        buffer[index] = '\0';

        printf("\b \b");
        fflush(stdout);
    }

    continue;
}
```

```c
/* Escape sequence */
if (ch == 27) {
    int ch2 = getchar();

    if (ch2 == '[') {
        int ch3 = getchar();
```

```c
        /* UP arrow */
        if (ch3 == 'A') {
            if (history_index > 0) {
                history_index--;

                while (index > 0) {
                    printf("\b \b");
                    index--;
                }

                strcpy(buffer, history[history_index]);
                index = strlen(buffer);

                printf("%s", buffer);
                fflush(stdout);
            }
        }

        /* DOWN arrow */
        else if (ch3 == 'B') {
            if (history_index < history_count - 1) {
                history_index++;

                while (index > 0) {
                    printf("\b \b");
                    index--;
                }

                strcpy(buffer, history[history_index]);
                index = strlen(buffer);

                printf("%s", buffer);
                fflush(stdout);
```

```
            } else {
                history_index = history_count;

                while (index > 0) {
                    printf("\b \b");
                    index--;
                }

                buffer[0] = '\0';
            }
        }
    }

    continue;
}
```

```
            /* Normal character */
            if (index < MAX_BUFFER - 1) {
                buffer[index++] = ch;
                buffer[index] = '\0';

                putchar(ch);
                fflush(stdout);
            }
        }
    }

    restore_mode(&old);
    return 0;
}
```

OUTPUT:

```
root@DUDDUKURIs: ~/skill/w

root@DUDDUKURIs:~# cd skill/week-3
root@DUDDUKURIs:~/skill/week-3# nano main.c
root@DUDDUKURIs:~/skill/week-3# gcc main.c -o main
root@DUDDUKURIs:~/skill/week-3# ./main
Session 3 - Command History
Type commands. Use UP/DOWN arrows for history.
Press Ctrl+C to stop.

myshell> hello world
Command: hello world
myshell>
[1]+  Stopped                 ./main
root@DUDDUKURIs:~/skill/week-3#
```

Task 2: To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.
CODE:

root@DUDDUKURIs: ~/skill/w

GNU nano 7.2 main2.c *

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_SIZE 10

typedef struct Node {
    char *command;
    struct Node *next;
} Node;

int main() {
    char *buffer;
    size_t size = INITIAL_SIZE;
    size_t length = 0;
    int ch;

    Node *head = NULL;
    Node *tail = NULL;

    buffer = malloc(size);

    if (buffer == NULL) {
        perror("malloc failed");
        return 1;
    }

    printf("Session 3 - Dynamic Memory Management\n");
    printf("Enter commands. Type 'exit' to finish.\n\n");
```

```c
while (1) {
    length = 0;

    printf("myshell> ");
    fflush(stdout);

    while ((ch = getchar()) != '\n' && ch != EOF) {

        if (length + 1 >= size) {
            size *= 2;

            char *temp = realloc(buffer, size);

            if (temp == NULL) {
                perror("realloc failed");
                free(buffer);
                return 1;
```

```
            }

            buffer = temp;
        }

        buffer[length++] = ch;
    }

    buffer[length] = '\0';

    if (strcmp(buffer, "exit") == 0) {
        break;
    }
```

```
if (length == 0) {
    continue;
}

Node *new_node = malloc(sizeof(Node));

if (new_node == NULL) {
    perror("malloc failed");
    break;
}

new_node->command = malloc(length + 1);

if (new_node->command == NULL) {
    perror("malloc failed");
    free(new_node);
    break;
}

strcpy(new_node->command, buffer);
new_node->next = NULL;

if (head == NULL) {
    head = new_node;
    tail = new_node;
} else {
    tail->next = new_node;
    tail = new_node;
}
```

```c
        printf("Stored command: %s\n", buffer);
        printf("Buffer size: %zu bytes\n", size);
    }

    free(buffer);

    Node *current = head;

    while (current != NULL) {
        Node *next = current->next;

        free(current->command);
        free(current);

        current = next;
    }

    printf("\nAll dynamically allocated memory released.\n");

    return 0;
}
```

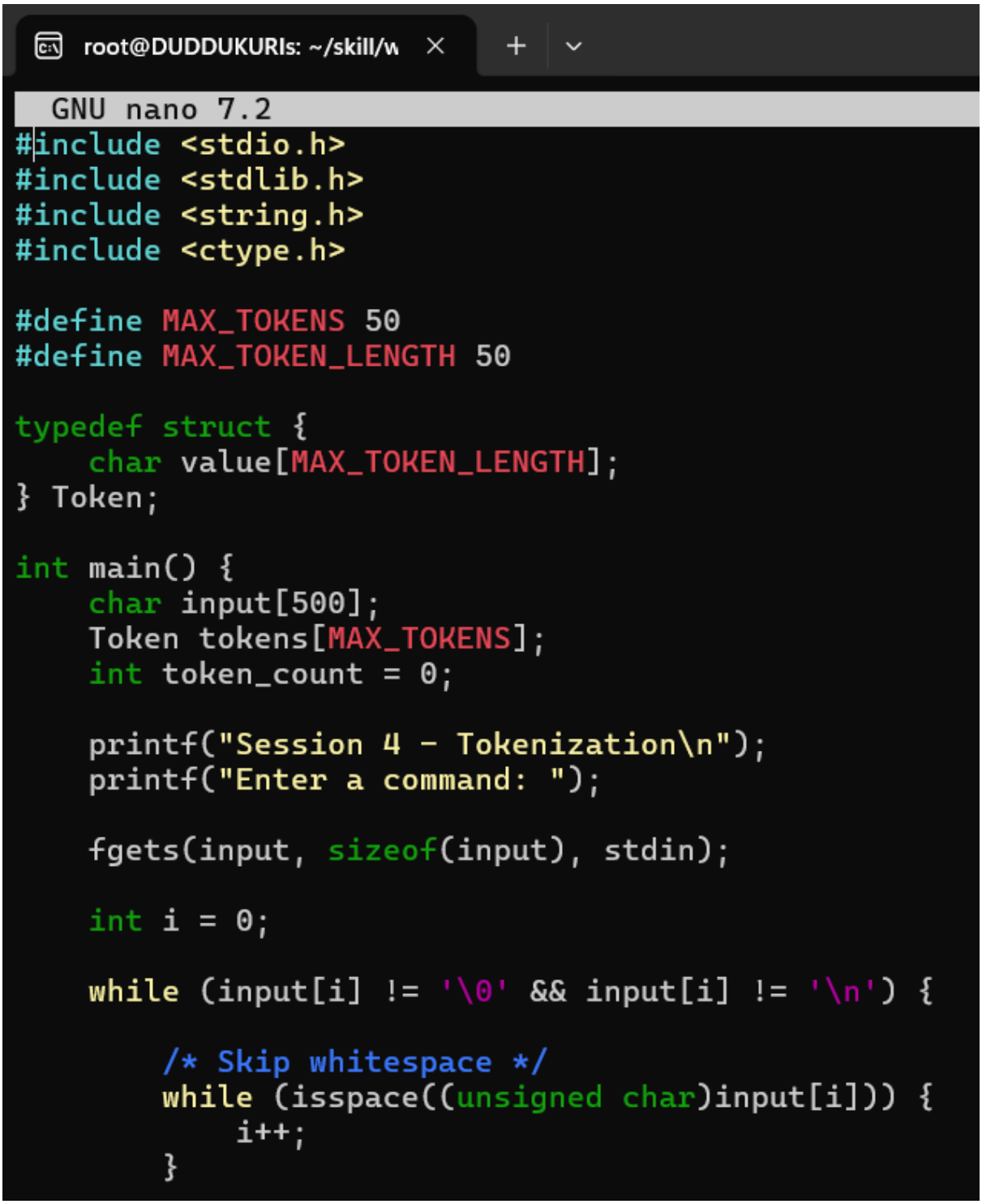
OUTPUT:

```
root@DUDDUKURIs: ~/skill/w
root@DUDDUKURIs:~# cd skill/week-3
root@DUDDUKURIs:~/skill/week-3# nano main2.c
root@DUDDUKURIs:~/skill/week-3# gcc main2.c -o main2
root@DUDDUKURIs:~/skill/week-3# ./main2
Session 3 - Dynamic Memory Management
Enter commands. Type 'exit' to finish.

myshell> hello
Stored command: hello
Buffer size: 10 bytes
myshell>
```

Task 1: To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output

CODE:



```
if (input[i] == '\0' || input[i] == '\n')
    break;

/* Check token limit */
if (token_count >= MAX_TOKENS) {
    printf("Error: Too many tokens.\n");
    break;
}

int j = 0;

/* Read characters until whitespace or delimiter */
while (input[i] != '\0' &&
       input[i] != '\n' &&
       !isspace((unsigned char)input[i]) &&
       input[i] != '|') {

    if (j < MAX_TOKEN_LENGTH - 1) {
        tokens[token_count].value[j++] = input[i];
    }

    i++;
}

tokens[token_count].value[j] = '\0';

if (j > 0) {
    token_count++;
}
```

```
    /* Handle delimiters */
    if (input[i] == '|' || input[i] == '|') {

        if (token_count >= MAX_TOKENS) {
            printf("Error: Too many tokens.\n");
            break;
        }

        tokens[token_count].value[0] = input[i];
        tokens[token_count].value[1] = '\0';

        token_count++;
        i++;
    }
}
```

```c
    printf("\nTokens identified:\n");

    if (token_count == 0) {
        printf("No tokens found.\n");
        return 0;
    }

    for (int k = 0; k < token_count; k++) {
        printf("Token %d: [%s]\n", k + 1, tokens[k].value);
    }

    printf("\nToken stream validation:\n");
```

```c
    int valid = 1;

    for (int k = 0; k < token_count; k++) {
        if (strlen(tokens[k].value) == 0) {
            valid = 0;
        }
    }

    if (valid)
        printf("Token stream is valid.\n");
    else
        printf("Token stream is invalid.\n");

    return 0;
}
```

OUTPUT:

```
root@DUDDUKURIs:~/skill/week-4# nano main.c
root@DUDDUKURIs:~/skill/week-4# gcc main.c -o main
root@DUDDUKURIs:~/skill/week-4# ./main
Session 4 - Tokenization
Enter a command: ls

Tokens identified:
Token 1: [ls]

Token stream validation:
Token stream is valid.
root@DUDDUKURIs:~/skill/week-4#
```

Task 2: To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

CODE:

root@DUDDUKURIs: ~/skill/w

GNU nano 7.2 main2.c *

```c
#include <stdio.h>
#include <string.h>

#define MAX_TOKENS 50

int main() {
    char input[500];
    char *tokens[MAX_TOKENS];
    int token_count = 0;
    int valid = 1;

    printf("Session 4 - Parser\n");
    printf("Enter a command: ");

    fgets(input, sizeof(input), stdin);

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    /* Handle empty command */
    if (strlen(input) == 0) {
        printf("Empty command detected.\n");
        return 0;
    }

    /* Tokenize input */
    char *token = strtok(input, " \t");
```

```c
while (token != NULL && token_count < MAX_TOKENS) {
    tokens[token_count++] = token;
    token = strtok(NULL, " \t");
}

printf("\nParse Structure:\n");

if (token_count == 0) {
    printf("No tokens found.\n");
    return 0;
}

/*
 * Basic syntax validation:
 * A pipe cannot be the first or last token.
 * Two pipes cannot occur consecutively.
 */
```

```
for (int i = 0; i < token_count; i++) {

    if (strcmp(tokens[i], "|") == 0) {

        /* Pipe at beginning or end */
        if (i == 0 || i == token_count - 1) {
            valid = 0;
            printf("Syntax Error: Invalid pipe position.\n");
        }
```

```
        /* Consecutive pipes */
        if (i > 0 && strcmp(tokens[i - 1], "|") == 0) {
            valid = 0;
            printf("Syntax Error: Consecutive pipes detected.\n");
        }
    }
}

if (!valid) {
    printf("\nParsing failed.\n");
    return 1;
}

/* Display execution structure */
printf("Command: %s\n", tokens[0]);

if (token_count > 1) {
    printf("Arguments:\n");

    for (int i = 1; i < token_count; i++) {
        if (strcmp(tokens[i], "|") != 0) {
            printf("  Argument %d: %s\n", i, tokens[i]);
        }
    }
}
```

```
    /* Display parse tree / execution structure */
    printf("\nParse tree / execution structure:\n");
    printf("ROOT\n");
    printf("  └── COMMAND: %s\n", tokens[0]);

    for (int i = 1; i < token_count; i++) {

        if (strcmp(tokens[i], "|") == 0) {
            printf("      └── PIPE\n");
        } else {
            printf("      └── ARGUMENT: %s\n", tokens[i]);
        }
    }

    printf("\nSyntax validation: SUCCESS\n");
    printf("Execution structure generated successfully.\n");

    return 0;
}
```

```
root@DUDDUKURIs:~/skill/week-4# nano main2.c
root@DUDDUKURIs:~/skill/week-4# gcc main2.c -o main2
root@DUDDUKURIs:~/skill/week-4# ./main2
Session 4 - Parser
Enter a command: ls -l | grep txt

Parse Structure:
Command: ls
Arguments:
  Argument 1: -l
  Argument 3: grep
  Argument 4: txt

Parse tree / execution structure:
ROOT
  └── COMMAND: ls
      └── ARGUMENT: -l
      └── PIPE
      └── ARGUMENT: grep
      └── ARGUMENT: txt

Syntax validation: SUCCESS
Execution structure generated successfully.
root@DUDDUKURIs:~/skill/week-4#
```